



Find Majority Element in an Array

Problem Statement

- Given an array of size n , find the element which occurs more than $n/2$ times. This element is called Majority Element.
- At the most, there will be only one majority element.
- If not found, return null.

Example 1

Array

2	6	2	2	6	2	2	8	2	1
---	---	---	---	---	---	---	---	---	---

Length of array, $n = 10$

Majority Element = Element which occurs more than $n/2 = 5$ times

Example 1

Array

2	6	2	2	6	2	2	8	2	1
---	---	---	---	---	---	---	---	---	---

Length of array, $n = 10$

Majority Element = Element which occurs more than $n/2 = 5$ times

Solution = 2



Example 2

Array

1	7	8	2	6	8	1	3	2	8
---	---	---	---	---	---	---	---	---	---

Length of array, $n = 10$

Majority Element = Element which occurs more than $n/2 = 5$ times

No majority element found!

Algorithm 1

Use 2 loops

Algorithm 1

Use 2 loops -

- In inner loop calculate the count of every element
- In outer loop check, if count is greater than $n/2$, return the element.

If out of the loops, return null.

Time Complexity: $O(n^2)$

Algorithm 2

Use Sorting

Algorithm 2

1. Sort the array
2. Again iterate over the array once and count occurrence of each element.
3. If an element is found with count $> n/2$, return the element.
4. If no such element found, return null.

Time Complexity:

1. If $O(n \log n)$ sorting algorithm is used, sorting step takes $O(n \log n)$ time
2. Counting step takes $O(n)$ time

Hence time complexity is $O(n \log n)$

Algorithm 3

Use Boyer-Moore Vote Algorithm

Algorithm 3

Step 1: Find a candidate for majority element.

Step 2: Check if this candidate is a majority element.

Time Complexity: $O(n)$

Algorithm 3

Step 1:

Find the candidate for majority element

1. Initialize count of current candidate as 0 $\rightarrow$ count = 0
2. Iterate over the array and do following steps:
 - a) If count == 0, set candidate = array[i], count = 1
 - b) Else
 - If candidate == array[i],
set count = count + 1
 - else set count = count - 1

Algorithm 3

Step 2:

Check if candidate is Majority Element

1. If $\text{count} == 0$, there is no majority element.
2. Else, iterate over array to get count of candidate.
 1. If $\text{count} > n/2$, return candidate
 2. Else return null;

Algorithm 3: Example

Array

n = 10



Step 1:

count = 2 candidate = 4

```
// Step 1: Find max element
Integer candidate = null;
int count = 0;
for(int i = 0; i < array.length; i++) {
    if(count == 0) {
        candidate = array[i];
        count = 1;
        continue;
    } else {
        if(candidate == array[i]) {
            count++;
        } else {
            count--;
        }
    }
}
```

Algorithm 3: Example

Array

4	7	4	4	7	4	4	9	4	3
---	---	---	---	---	---	---	---	---	---

n = 10

Step 2: Check if candidate is Majority Element candidate = 4

count = 6

Return 4

```
if(count == 0) {  
    return null;  
}  
  
// Step 2: Check if candidate is majority element  
count = 0;  
for(int i = 0; i < array.length; i++) {  
    if(candidate == array[i]) {  
        count++;  
    }  
}  
return (count > array.length/2) ? candidate : null;
```

Why do we need Step 2?

candidate obtained from step 1, may not be a majority element. For example:

i	0	1	2	3	4
Array	2	1	2	4	7

After step 1, candidate = 7, count = 1

Clearly, 7 is not majority element

Order of the Algorithm 3

Time Complexity: $O(n)$

Space Complexity: $O(1)$

public class MajorityElement {	21	for (int num : nums) {
2	22	if (num == candidate) {
3	23	majorityCount++;
findMajorityElement(int[] nums) {	24	}
4	25	}
5	26	
6	27	if (majorityCount > nums.length / 2) {
7	28	return candidate;
8	29	}
9	30	
10	31	return -1; // No majority element found
11	32	}
12	33	
13	34	public static void main(String[] args) {
14	35	int[] nums = {3, 2, 3};
15	36	int majorityElement = findMajorityElement(nums);
16	37	
17	38	if (majorityElement != -1) {
18	39	System.out.println("Majority element: " + majorityElement)
19	40	} else {
20	41	System.out.println("No majority element found.");
	42	}
	43	}
	44	}



THANK YOU